How To Measure Clock Frequency



SURF VHDL

THE EASIEST WAY TO LEARN VHDL

Measure the Clock Frequency in Your FPGA using VHDL

TABLE OF CONTENTS

INTRODUCTION 3

WHAT DO YOU NEED? 4

THE FIRST CHECK YOU SHOULD DO 6

HOW CAN WE DETECT THE CLOCK? 8

COMPUTE THE CLOCK FREQUENCY 9

VHDL IMPLEMENTATION OF THE CLOCK FREQUENCY COUNTER 12

CLOCK DOMAIN CROSSING STRATEGY 16

VHDL CODE SIMULATION AND FPGA IMPLEMENTATION 17

LICENSE 19

INTRODUCTION

Hi,

I'm Francesco Richichi and I want to thank you or downloading this eBook.

Just few words to introduce myself.

I'm an Electronic Engineer and I worked in Digital design since 1997.

I developed a lot of HW design both in ASIC and FPGA:

High Speed MODEM, Turbo Encoder/Decoder, LDPC Encoder/Decoder, Radar processing.

In the middle of 2015 I decided to start the website "**http://surf-vhdl.com/**" because I think is the moment to give back some of my knowledge to whom need to know about VHDL.

When I started, I had to study from books and no tutorial were available on the internet. Fortunately, now is completely different. You can find all the info you need for any topic and you can find lot of information for free!

But there is the other side of the coin: you can find a lot of useless stuff!

Even if you don't think about if, useless stuff is a big loss of money.

Yes, MONEY.

Every time you have to understand if the info you found is useful or not you are using your time.

TIME = MONEY, more TIME > MONEY!

Using your time in a proper way you can get money, the vice versa is not always true.

What do you think about it?

I hope I'll provide you useful information and let's start learning how to implement your first VHDL design in FPGA

WHAT DO YOU NEED?

The exercises are implemented using VHDL so you should have a

- VHDL editor
- VHDL simulator
- Board equipped with an FPGA to test the design (optional but recommended)

As VHDL editor on the internet you can find a lot of text editor with syntax highlight. You can use Textpad (http://www.textpad.com), Notepad++ (https://notepad-plus-plus.org) for windows user, TextWrangler for Mac user. These text editors are free.

As VHDL simulator, in the course we will use ModelSim that is one of the most powerful VHDL/Verilog simulator.

ModelSim license is very expensive but you can get a free license legally, next I'll show how...

As layout tool, we will use Altera Quartus II. The test of the VHDL code on FPGA will be performed on an Altera DE0 board.

The Quartus II license are very expensive too…

Maybe you are thinking… ok the eBook is free but I should spend a lot of money buying license!

Don't worry, I'm here to help you.

Using the Altera Quartus II edition provided with the DE0 board you can get a free "lite" license both for ModelSim and Quartus II. "Lite" means that you can simulate and layout only a subset of Altera devices. This is enough because the you can simulate and layout all the "non-professional" (very expensive and very huge) FPGA devices, i.e. you can handle all the FPGA you can afford to buy.

OK… at this point you are asking: if I don't have the DE0 board?

I'm still here to resolve this problem: go on Altera website, register and download the **Quartus II web edition**. You can get the full Altera suite with ModelSim simulator that you can use to implement your VHDL design.

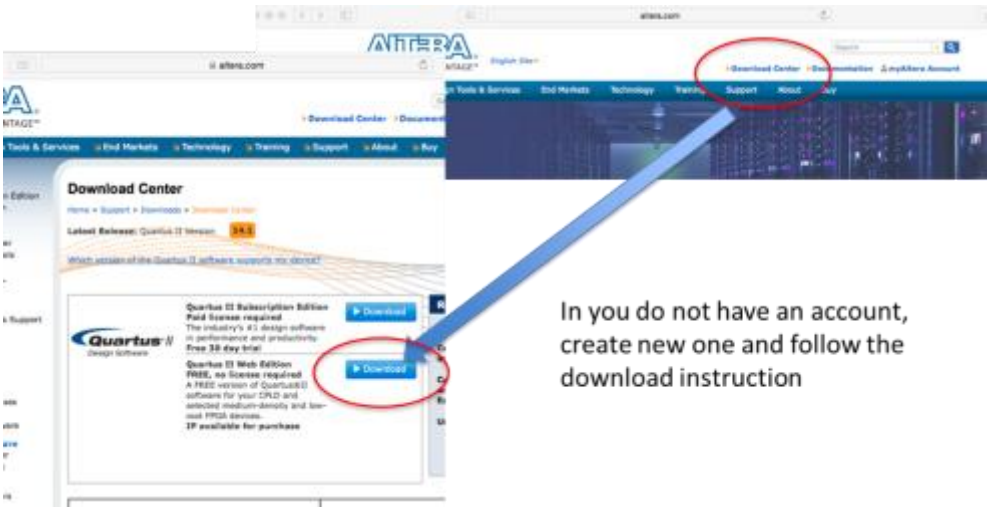


Figure 1 Altera website where download Quartus II

THE FIRST CHECK YOU SHOULD DO

When you use an FPGA you always need a clock. I mean, you can design an FPGA without the need of a clock but in this case, you are implementing a huge combinatorial logic… honestly I don't think it is so useful.

Let's assume we are trying to design a VHDL design on FPGA respecting at least the basic rules of the good VHDL designer:

- Synchronous design;
- Well defined clock domains;
- Register all the input and output of your FPGA;
- Avoid long combinatorial path;

Of course, the checklist is quite longer, but if you start to respect these steps you are starting with the right foot.

So, let's assume you have respected these basic rules, and design your FPGA. Write your VHDL code and, as clever designer, simulate and layout it.

Program the FPGA and start testing your VHDL code on your board. Often your design doesn't work as it should! Does it sound familiar?

It's the hardware my friend!

It is true that digital design has a big advantage w.r.t analog design:

if you implement a (good) synchronous design and simulate it, you are very confident that the design can work as it should.

But the reality is different!

I have a bad news when you start to debug your design on FPGA it always doesn't work... Did it happen to you?

There is also good news… If you did a good simulation of your digital VHDL design, often the problems you get during the test on a board are of the same type:

1) you didn't handle the reset pin;
2) the clock signal is absent or of wrong value

Maybe you are thinking... it is not possible that the main errors are the clock and reset!

Well, if you did simulate your VHDL design, and you did respect the golden rules for a digital design external reset and clock are often the main issue of your design!

Seeing is believing.

HOW CAN WE DETECT THE CLOCK?

For the reset pin, the solution is very simple: you can map reset to an output pin connected to a LED so you can check if the reset signal is working fine...

If you don't have the possibility to drive a LED, just put a multimeter on the reset pin and check for the right behavior. Reset has a static value.

What about the clock?

Here the issue is a bit different. The clock is a periodic signal changing very fast, so you cannot directly connect the clock to a LED but.... if you divide the clock rate to a "human" frequency you can use it to blink a LED.



Figure 2 Clock signal example

I mean, connect the clock to a module that implements a counter to generate a slow signal that can drive a LED for instance at the toggling rate of one second as in Figure 3. Doing this you can monitor if your clock is present.

It is a good solution but.... How can you measure the clock frequency? You can guess something from the blinking rate but is only a rough estimation.

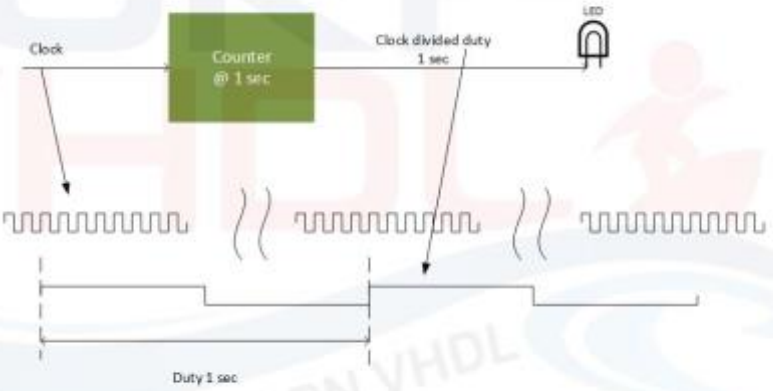


Figure 3 clock divider connected to an external LED

COMPUTE THE CLOCK FREQUENCY

So, we find a solution to detect if the clock is present but we can't have a feedback of the clock frequency.

You can guess: trivial, use an oscilloscope and measure all the clock parameters.

Right! If you have an oscilloscope the solution is fine, but if you can't afford the price of an oscilloscope? Or we don't want to use it?

The basic idea is to implement a clock counter, and count the number of clock cycle during a known interval of time. The architecture is highlighted in Figure 4.

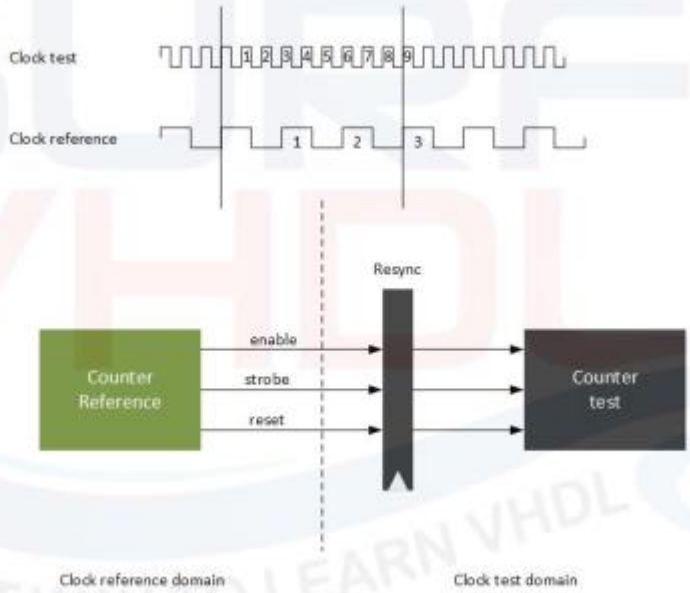


Figure 4 Clock period counter architecture

As you can see, we use a reference clock to detect a test clock.

The reference clock counts for a fixed number clock cycles. This defines a time interval. For instances if the test clock is 50 MHz, i.e. 20 ns, the counting value is 2^{12}, counting time period is:

20 ns * 2^{12} = 81.920ns = 81,920 us

In order to compute the test clock frequency, we need to know how many clock cycles of the test clock are present into the reference interval. So, we can use the reference clock counter to start and stop the clock test counter.

Maybe an example should clarify:

REF clock = 50 MHz

REF clock count = 2^{12} = 4096 clock cycles

TEST clock = 125 MHz

every 4096 reference clock cycles, the test clock counts for 10240 as in simulation reported in Figure3

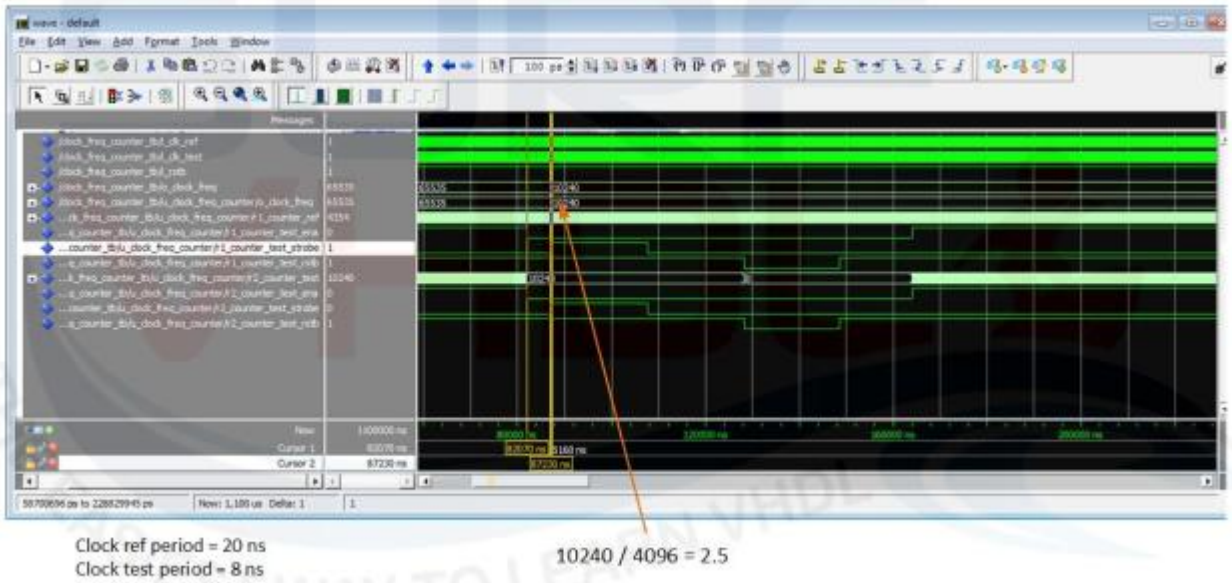


Figure 5 VHDL code clock counter simulation with test clock 125 MHz

The test clock frequency will be:

10240/4096* 50 MHz = 2.5*50 = 125 MHz

a second example, if test clock counter counts for 2048

The test clock frequency will be: 2048/4096* 50 = 0.5 * 50 = 25 MHz (40 ns) as in simulation reported in Figure 6.

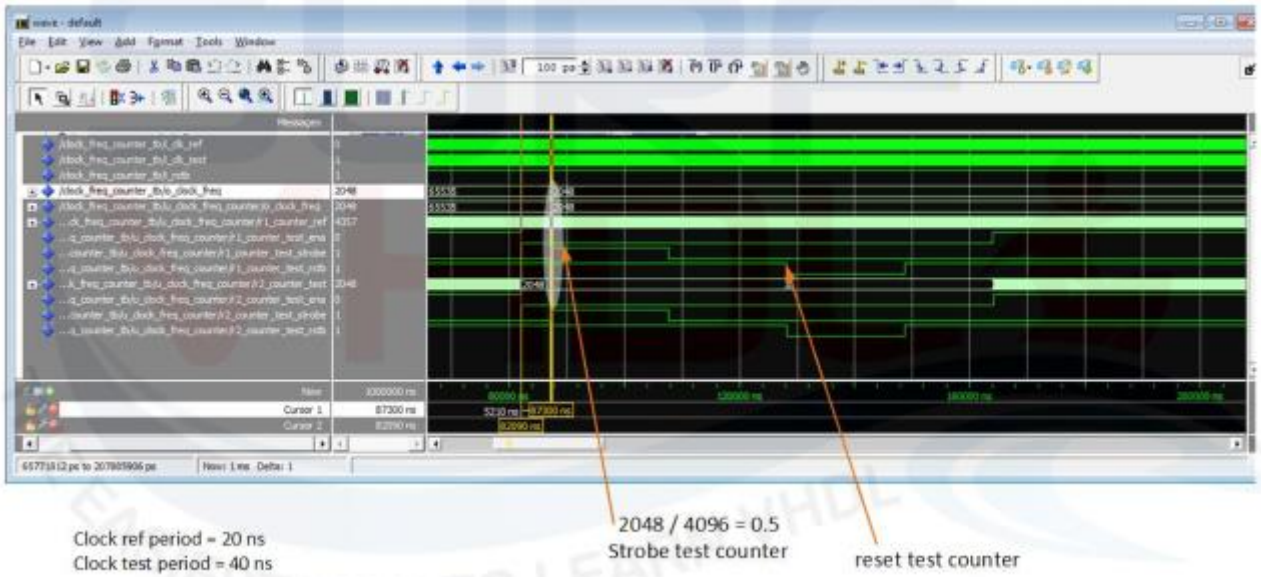


Figure 6 VHDL code clock counter simulation with test clock 25 MHz

VHDL IMPLEMENTATION OF THE CLOCK FREQUENCY COUNTER

As we saw, the basic idea in clock frequency computing is very simple, but we have to pay attention in implementing the VHDL component.

As you can see the two clocks, reference clock, and test clock, are asynchronous.

When we have to handle two different asynchronous clock domains we need to pay attention in clock domain crossing (CDC)

In Figure 4, it should be clear that the control signal for the test counter is resynchronized in the test clock domain in order to control the counter test. If you do not perform such re-synchronization the layout tool couldn't implement the correct control logic for the clock test domain.

A golden rule you should remember when you have to cross a clock domain is to perform the clock domain crossing using a single signal: a single wire NO a set of wires.

If you use more than a signal you can have different delays (skew between the signals) and you can strobe them at different clock edge in the new domain. Using a single signal the value you strobe is always correct by definition ‘0’ or ‘1’.

```
47
48  library ieee;
49  use ieee.std_logic_1164.all;
50  use ieee.numeric_std.all;
51
52  entity clock_freq_counter is
53  port (
54      i_clk_ref            : in  std_logic;
55      i_clk_test           : in  std_logic;
56      i_rstb               : in  std_logic;
57      o_clock_freq         : out std_logic_vector(15 downto 0));
58  end clock_freq_counter;
59
60  architecture rtl of clock_freq_counter is
61
62  -- r1_  register con clock reference domain
63  -- r2_  register con clock test domain
64
65  --  CLOCK REFERENCE signal declaration
66  signal r1_counter_ref              : unsigned(12 downto 0);  -- 12+1 bit: extra bit used for test counter control
67  signal r1_counter_test_ena         : std_logic;
68  signal r1_counter_test_strobe      : std_logic;
69  signal r1_counter_test_rstb        : std_logic;
70  --  CLOCK TEST signal declaration
71  signal r2_counter_test             : unsigned(15 downto 0); -- clock test can be up-to 16 times clock ref
72  signal r2_counter_test_ena         : std_logic;
73  signal r2_counter_test_strobe      : std_logic;
74  signal r2_counter_test_rstb        : std_logic;
```

Figure 7 Clock frequency counter entity definition

In the VHDL code of Figure 7, the r1_ and r2_ prefixes denote the reference and test clock domain respectively.

```
begin
--  CLOCK REFERENCE domain
p_counter_ref : process (i_rstb,i_clk_ref)
begin
    if(i_rstb='0') then
        r1_counter_ref                <= (others=>'0');
        r1_counter_test_ena           <= '0';
        r1_counter_test_strobe        <= '0';
        r1_counter_test_rstb          <= '0';
    elsif(rising_edge(i_clk_ref)) then
        r1_counter_ref            <= r1_counter_ref + 1;  -- free running

        -- use MSB to control test counter
        r1_counter_test_ena       <= not r1_counter_ref(r1_counter_ref'high);

        -- enable storing for 1024 clock cycle after 256 clock cycle
        if(r1_counter_ref>16#1100#) and (r1_counter_ref<16#1500#) then
            r1_counter_test_strobe     <= '1';
        else
            r1_counter_test_strobe     <= '0';
        end if;

        -- enable reset for 1024 clock cycle; after 1024 clock cycle from storing
        if(r1_counter_ref>16#1900#) and (r1_counter_ref<16#1D00#) then
            r1_counter_test_rstb    <= '0';
        else
            r1_counter_test_rstb    <= '1';
        end if;

    end if;
end process p_counter_ref;
```

Figure 8 clock reference counter

As clear the "*p_clock_test_resync*" process in Figure 9 resynchronizes the control signal versus the test clock domain, and this guarantee the layout tool to work fine in the test clock domain.

```

p_clk_test_resync : process (i_clk_test)
begin
    if(rising_edge(i_clk_test)) then
        r2_counter_test_ena        <= r1_counter_test_ena     ;
        r2_counter_test_strobe     <= r1_counter_test_strobe  ;
        r2_counter_test_rstb       <= r1_counter_test_rstb    ;
    end if;
end process p_clk_test_resync;

p_counter_test : process (r2_counter_test_rstb,i_clk_test)
begin
    if(r2_counter_test_rstb='0') then
        r2_counter_test         <= (others=>'0');
    elsif(rising_edge(i_clk_test)) then
        if(r2_counter_test_ena='1') then
            r2_counter_test    <= r2_counter_test + 1;
        end if;
    end if;
end process p_counter_test;

p_counter_test_out : process (i_rstb,i_clk_test)
begin
    if(i_rstb='0') then
        o_clock_freq         <= (others=>'1');  -- set all bit to '1' at reset and if test clock is not present
    elsif(rising_edge(i_clk_test)) then
        if(r2_counter_test_strobe='1') then
            o_clock_freq         <= std_logic_vector(r2_counter_test);
        end if;
    end if;
end process p_counter_test_out;

end rtl;
```

Figure 9 clock test process

In the VHDL code for clock counter in Figure 8, the reference counter is set to 12 bit and the test counter is 16 bit: the test clock can run up to 16 times (16-12 bit: $2^4 = 16$) faster than the reference clock. If the test clock runs faster than 16 times, the 16-bit counter will wrap around and the frequency we detect the wrong frequency.

In this case, if we know the point we can compensate the wrap around in the clock computation equation adding 2^{16} to the clock test counter.

For instance, reference clock = 1 MHz, test clock = 18 MHz

Every 2^{12} clock reference cycles, the test clock counter will count

2^{12}*18 = = 4096*18 = 73728

but if we are using a 16 bit counter the value we get will be:

73728-65536 = 8192

In this case, if we do not take into account the counter wrapping the frequency we are computing will be

2^12/8192 * 1 MHz = 0.5 MHz

instead of 18 MHz!

CLOCK DOMAIN CROSSING STRATEGY

In the VHDL code of the clock frequency counter, we are using an extra bit on the reference counter as clear in line 66 of Figure 7. This trick is used to guarantee the correct clock domain crossing. As you can see, the strobe signal will be stable for 256 clock reference cycle (line 92 of Figure 8) then the reset signal will be asserted after 1024 clock reference cycles (line 99 of Figure 8) after the strobe signal de-assertion.

Using this strategy, we will assure that our clock frequency counter can work also with test clock slower than reference clock.

How slow?

In this implementation 256 times slower!

VHDL CODE SIMULATION AND FPGA IMPLEMENTATION

Here below you will find a link where are reported the simulation using different test condition:

- Test clock faster than reference
- Test clock slower than reference clock.

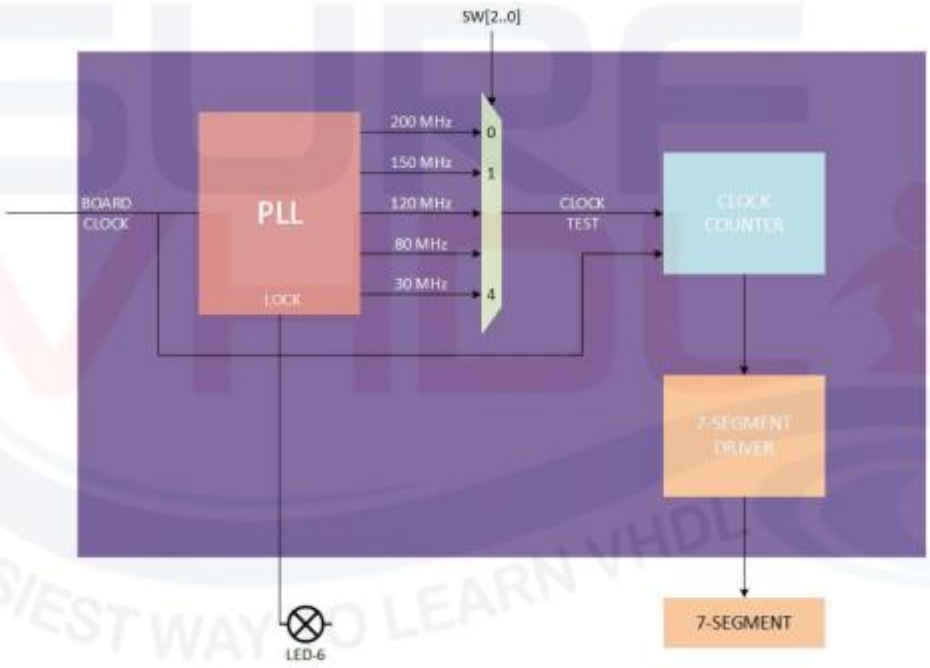


Figure 10 Clock counter test architecture on DE0 board

This design is implemented on a DE0 Altera board in order to verify the behavior on FPGA. In the FPGA design, we will use the internal PLL in order to generate different internal clocks selected by the board dip switch. You can find the complete simulation and demo in the free course

Measuring Clock Frequency in FPGA

Joining the course, you can download the VHDL source code of the design and try by yourself.

In the video you can review the concept we treated in this pages aided with a live explanation. Moreover, the design is played on an Altera DE0 board so you can verify the results on a live demo

Don’t forget to join SURF-VHDL on the social pages:



for any question write to:

surf.vhdl@gmail.com

LICENSE

Copyright (C) 2015 SURF-VHDL.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation;

with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.

A copy of the license is included in the section entitled

"GNU Free Documentation License".